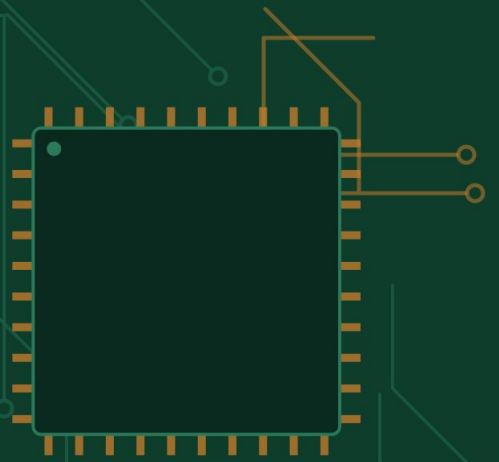
PERTEMUAN 05

Waktu dan Interupsi



Dari delay() yang membekukan program sampai timer perangkat keras

Sistem Tertanam (Embedded Systems) · S1 Teknik Elektro

Dosen Pengampu: Anton Prafanto

Tujuan Pembelajaran

- Menjelaskan masalah `delay()` dan menggantinya dengan pola `millis()`
- Menjalankan banyak tugas berkala dan memahami **luapan** `millis()`
- Membedakan *polling* dan **interupsi**, lalu memasang `attachInterrupt()`
- Menulis **ISR yang aman**: singkat, `IRAM_ATTR`, `volatile`, bagian kritis
- Memakai **timer perangkat keras** dan mengukur waktu dengan `micros()`

Peta Materi Hari Ini

01 Masalah delay()

02 millis(), micros(), dan luapan

03 Banyak tugas dalam satu loop()

04 Polling vs interupsi

05 Aturan ISR dan bagian kritis

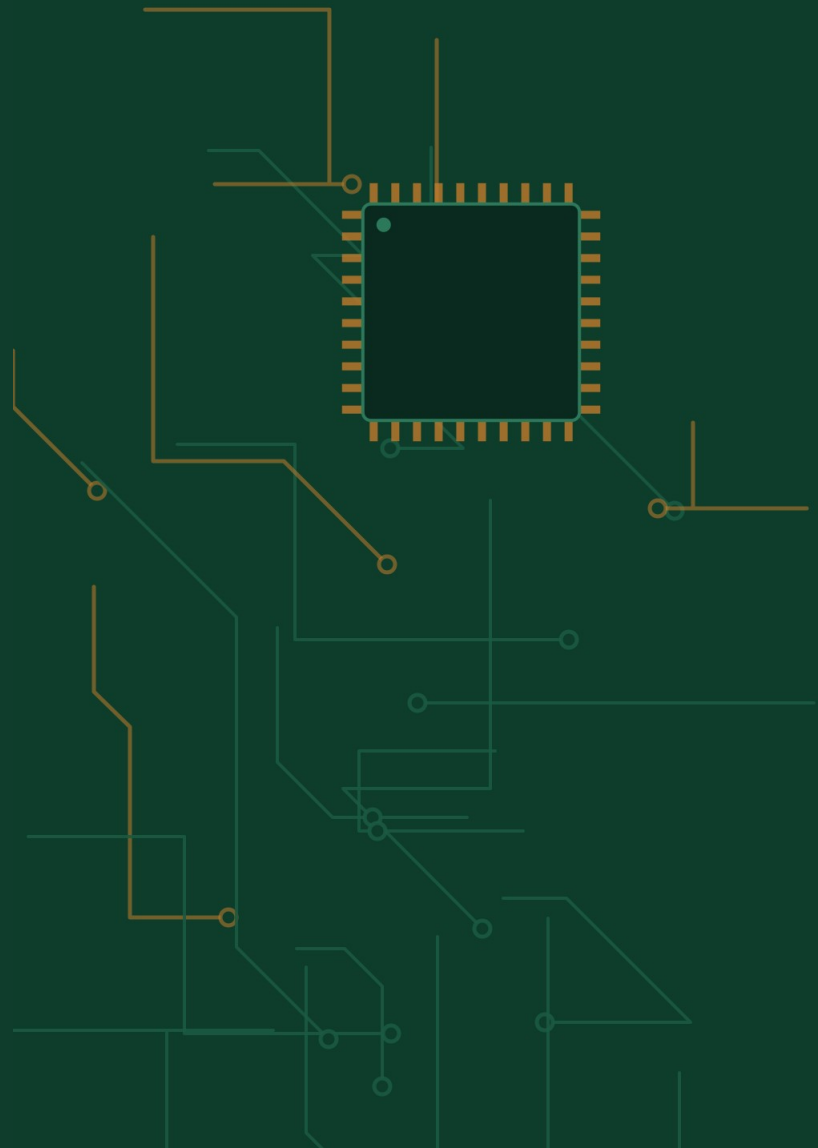
06 Timer perangkat keras

07 Praktikum 1–4

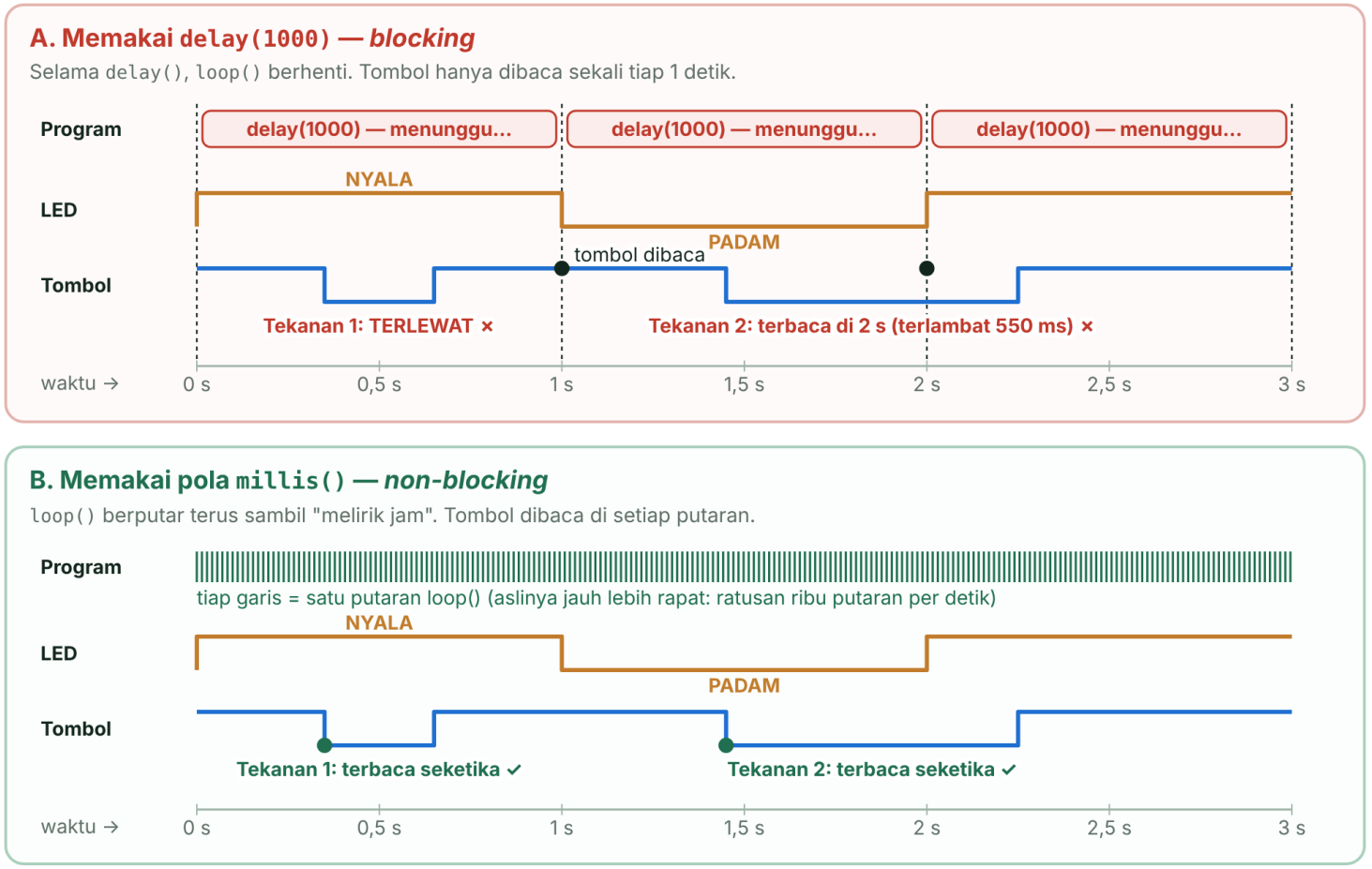
01

Waktu Tanpa Menunggu

Meninggalkan delay(), beralih ke millis()



delay() Membekukan Program



Pola "Sudah Lewat Waktunya?"

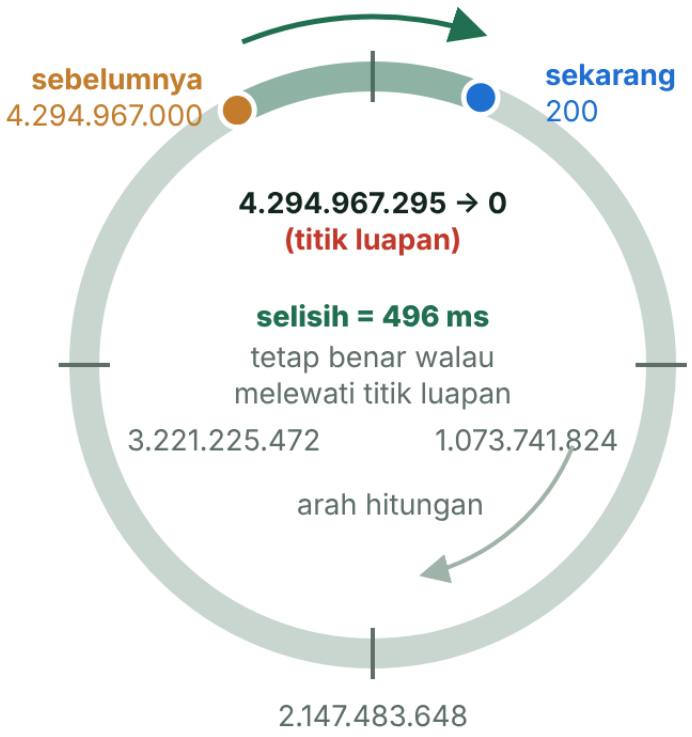
src/main.cpp

```cpp
unsigned long sebelumnya = 0;
const unsigned long INTERVAL = 1000;

void loop() {
  unsigned long sekarang = millis();
  if (sekarang - sebelumnya >= INTERVAL) {
    sebelumnya = sekarang;   // catat
    kerjakanTugas();         // balik LED
  }
  bacaTombol();              // tetap jalan
}
```

- `millis()` = milidetik sejak ESP32 menyala
- Belum waktunya → `if` dilewati dalam mikrodetik
- Simpan waktu di `unsigned long` (32 bit)
- Bandingkan **selisih**; jangan menunggu di `loop()`

Luapan millis(): Selisih Tetap Benar



Lingkaran = semua nilai 32-bit (jarak titik diperbesar).

✓ Benar: `sekarang - sebelumnya >= INTERVAL`

```
sebelumnya = 4.294.967.000   (sesaat sebelum luapan)
sekarang   =           200   (sesudah luapan)
sekarang - sebelumnya = 496 ms  ← selisih yang BENAR
```

Pengurangan `unsigned` otomatis "memutari lingkaran": 200 + 4.294.967.296 − 4.294.967.000 = 496. Tugas baru jalan saat selisih ≥ 1.000 ms.

× Salah: `sekarang >= sebelumnya + INTERVAL`

```
sebelumnya + 1000 = 4.294.968.000 → meluap menjadi 704
sekarang = 4.294.967.001  ≥  704 ?  → BENAR
```

Baru lewat 1 ms tetapi tugas sudah dijalankan, lalu terus berulang di setiap putaran sampai `millis()` ikut meluap.

±49,7 hari

`millis()`: 2^{32} ms = 49 hari 17 jam 2 menit 47 detik

±71,6 menit

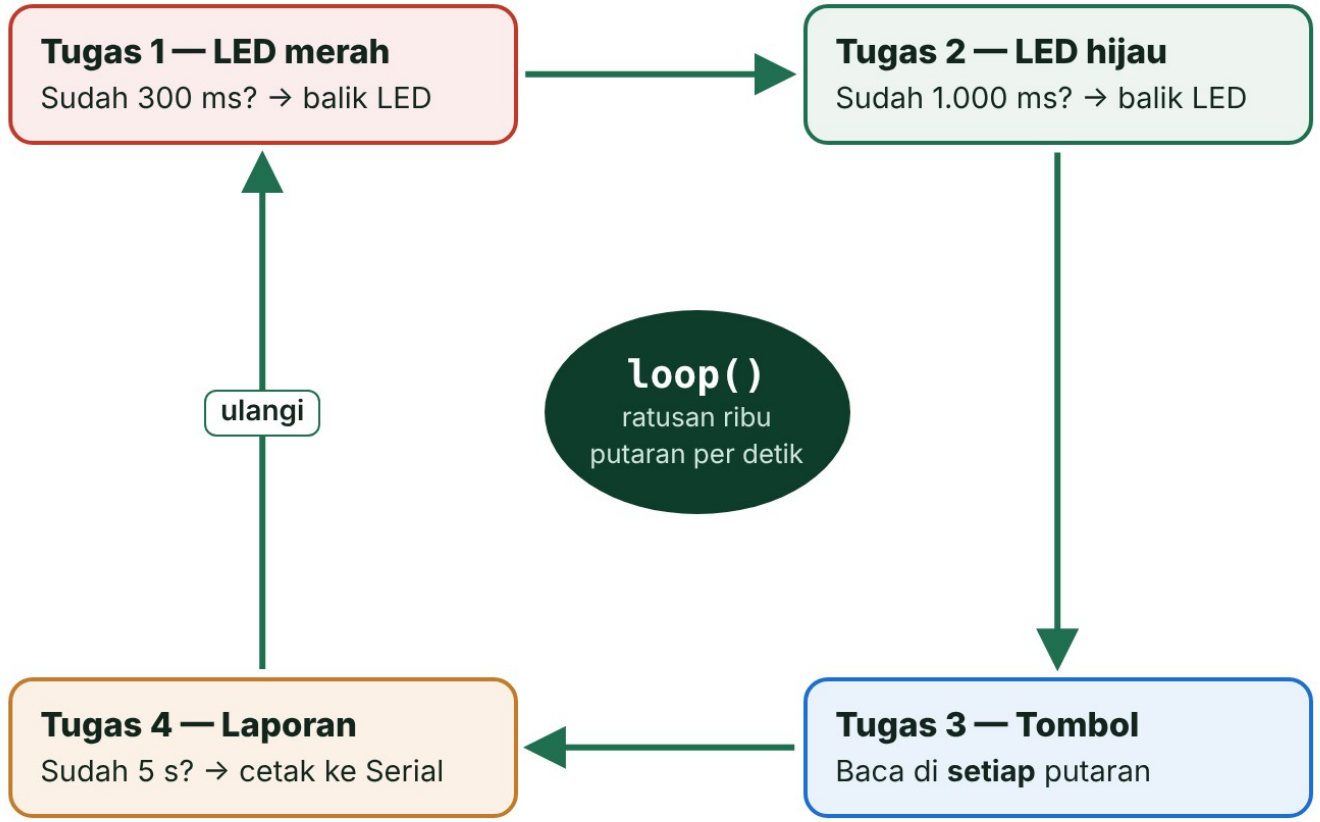
`micros()`: 2^{32} µs = 1 jam 11 menit 35 detik

Satu loop(), Banyak Tugas

Analogi dapur

- **Nasi**: cek setiap 20 menit
- **Telur**: angkat setelah 7 menit
- **Sayur**: aduk setiap 1 menit
- **Pintu**: segera dibuka bila ada tamu

Koki berkeliling terus. Setiap pekerjaan hanya butuh sebentar, jadi tidak ada yang telantar.

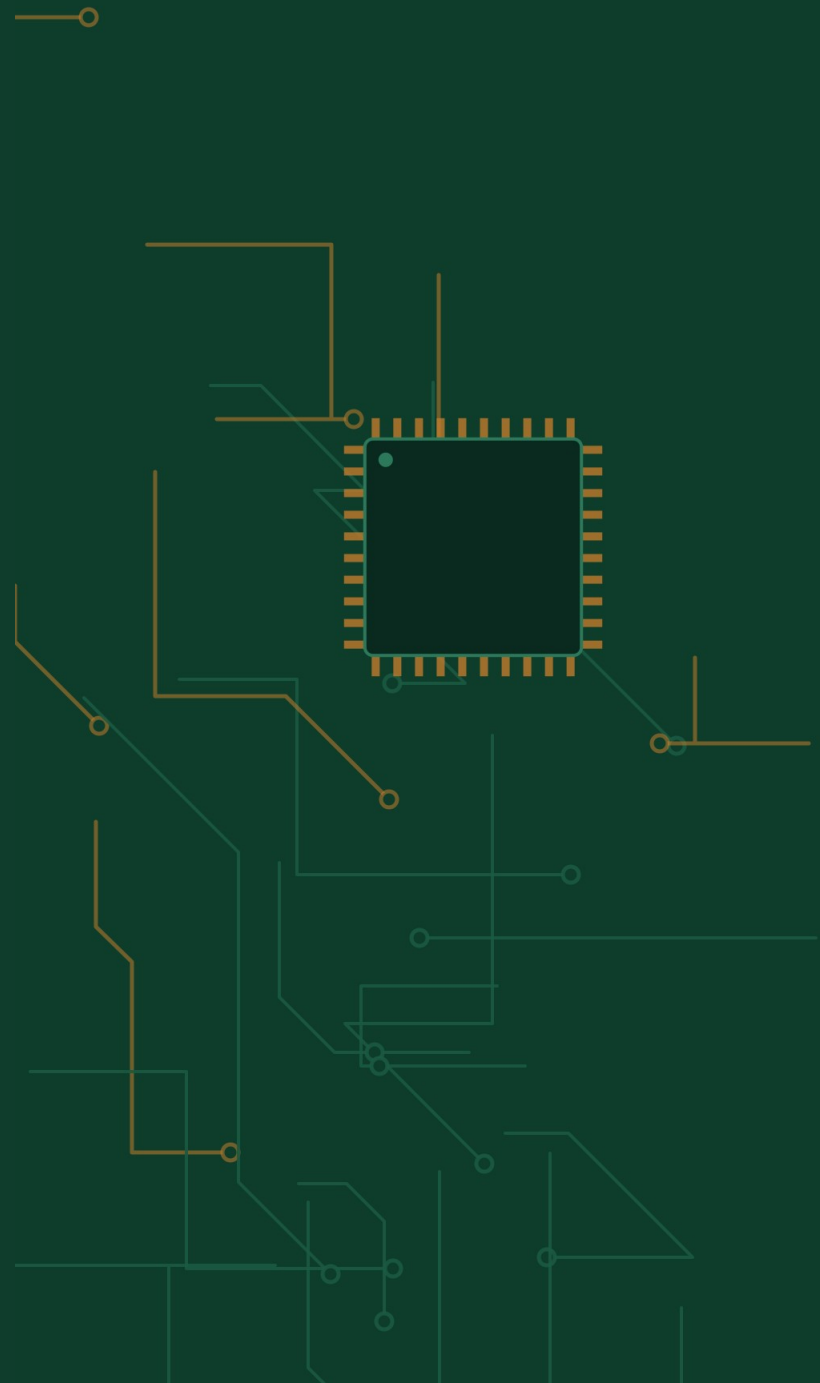


Ikon: Font Awesome Free (CC BY 4.0), Material Design Icons (Apache 2.0)

02

Interupsi

Biarkan perangkat keras membunyikan bel



Polling vs Interupsi



Polling

"Sudah ada tamu belum?" — dicek berulang-ulang



- Sederhana: cukup `digitalRead()` di dalam `loop()`.
- Waktu habis untuk mengecek walau tidak ada apa-apa.
- Bisa **terlambat**, bahkan **terlewat**, jika `loop()` lambat.



Interupsi

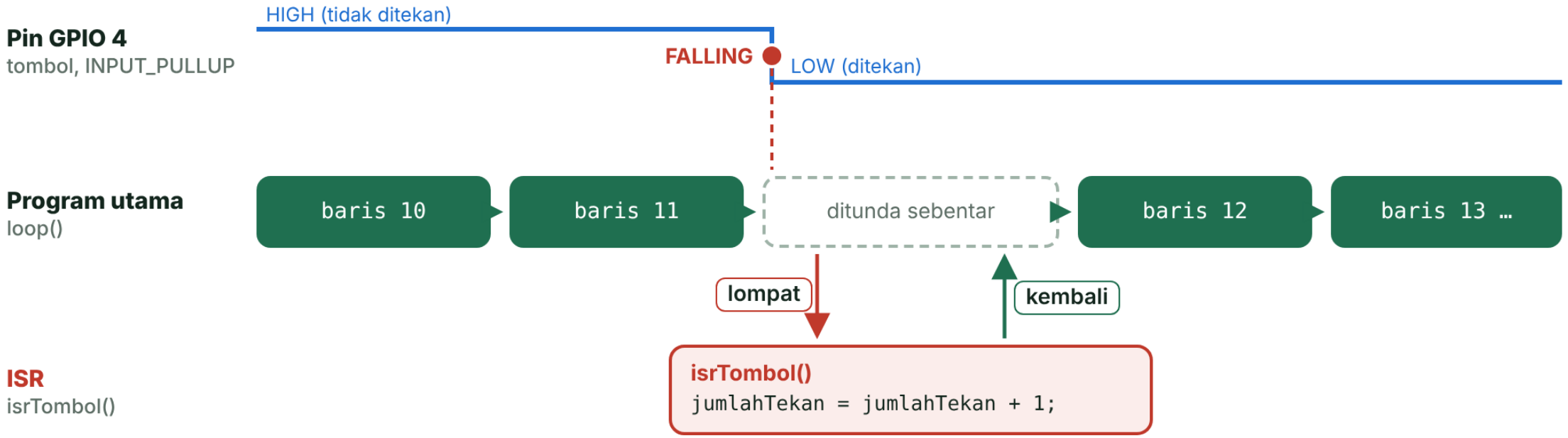
Bel berbunyi → hentikan sebentar → layani → lanjut



- Perangkat keras yang "membunyikan bel" saat tepi sinyal terjadi.
- Tanggapan hampir seketika (orde mikrodetik).
- CPU bebas mengerjakan hal lain; tidak ada tekanan yang terlewat.

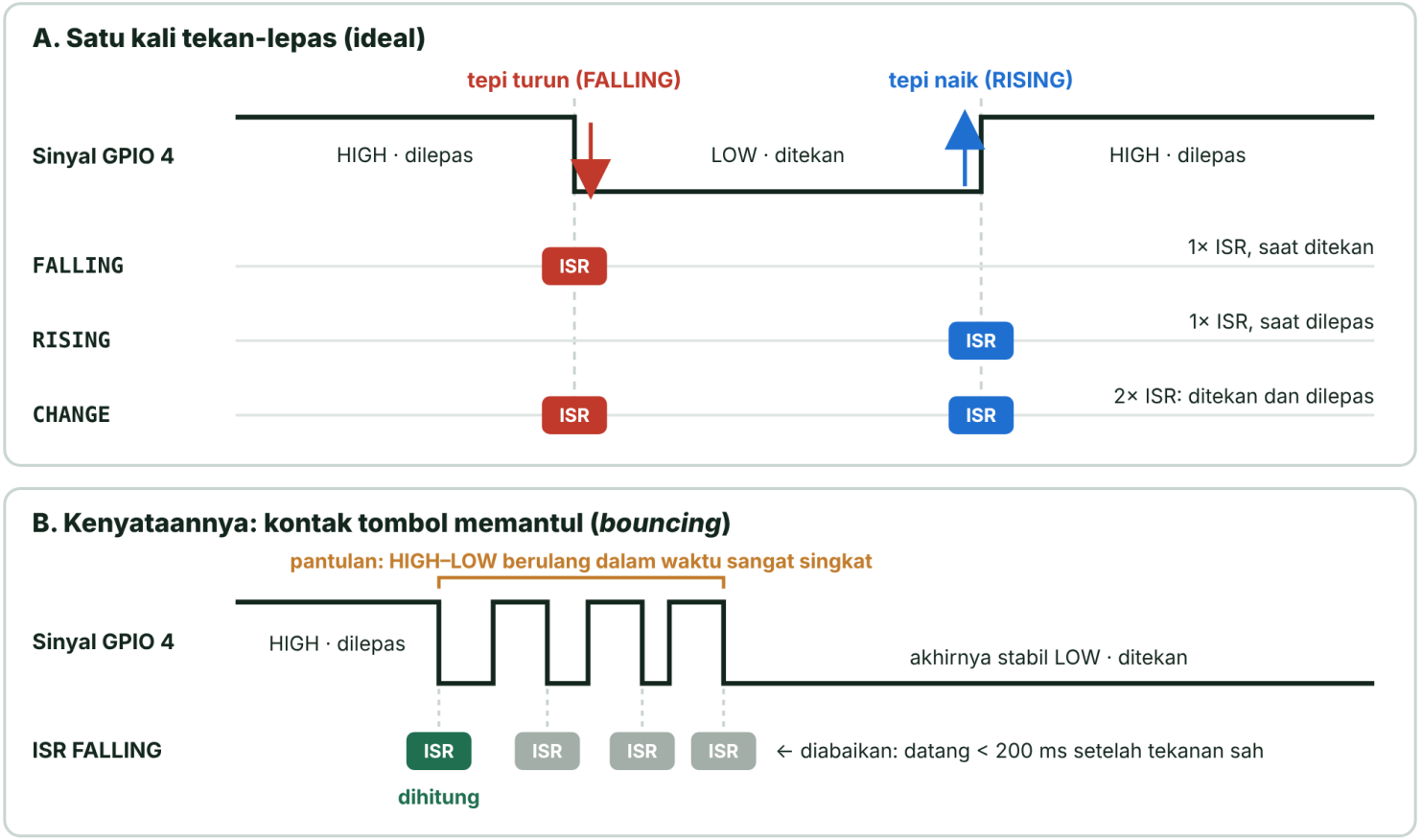
Ikon: Font Awesome Free (CC BY 4.0)

Apa yang Terjadi Saat Interupsi?



1. Tepi turun terdeteksi oleh perangkat keras GPIO.
2. CPU menyimpan "pembatas buku": posisi program dan isi register.
3. ISR dijalankan. Harus **singkat** (mikrodetik).
4. CPU memulihkan posisi dan melanjutkan baris 12.

Tepi Naik, Tepi Turun, dan Pantulan



Satu kali tekan bisa memicu beberapa interupsi. *Debounce* di ISR (Praktikum 2): terima interupsi pertama, abaikan yang datang kurang dari 200 ms sesudahnya.

Memasang Interupsi Tombol

kode/02-interupsi-tombol (disederhanakan)

```
volatile unsigned long jumlahTekan = 0;
volatile unsigned long waktuSah = 0;

void IRAM_ATTR isrTombol() {
  unsigned long t = millis();
  if (t - waktuSah >= 200) {  // debounce
    jumlahTekan = jumlahTekan + 1;
    waktuSah = t;
  }
}

void setup() {
  pinMode(4, INPUT_PULLUP);
  attachInterrupt(digitalPinToInterrupt(4),
                  isrTombol, FALLING);
}
```

- Pasang sekali di `setup()`
- Nama ISR ditulis **tanpa** tanda kurung
- Mode: `FALLING`, `RISING`, atau `CHANGE`
- `x = x + 1`: `++` pada `volatile` usang di C++ baru

Aturan Emas ISR

Singkat

Catat saja: penghitung, waktu, penanda. Sisanya di `loop()`.

IRAM_ATTR

Kode ISR di RAM internal, langsung siap dijalankan.

Tanpa Serial & delay()

Bisa *crash* (Guru Meditation) dan ESP32 me-*reset*.

volatile + bagian kritis

Data bersama selalu dibaca ulang dan diakses tanpa disela.

Bagian Kritis: Salin lalu Nolkan

pola ESP32 dua inti

```
portMUX_TYPE kunci = portMUX_INITIALIZER_UNLOCKED;

void IRAM_ATTR isrTombol() {
  portENTER_CRITICAL_ISR(&kunci);
  jumlahTekan = jumlahTekan + 1;
  portEXIT_CRITICAL_ISR(&kunci);
}

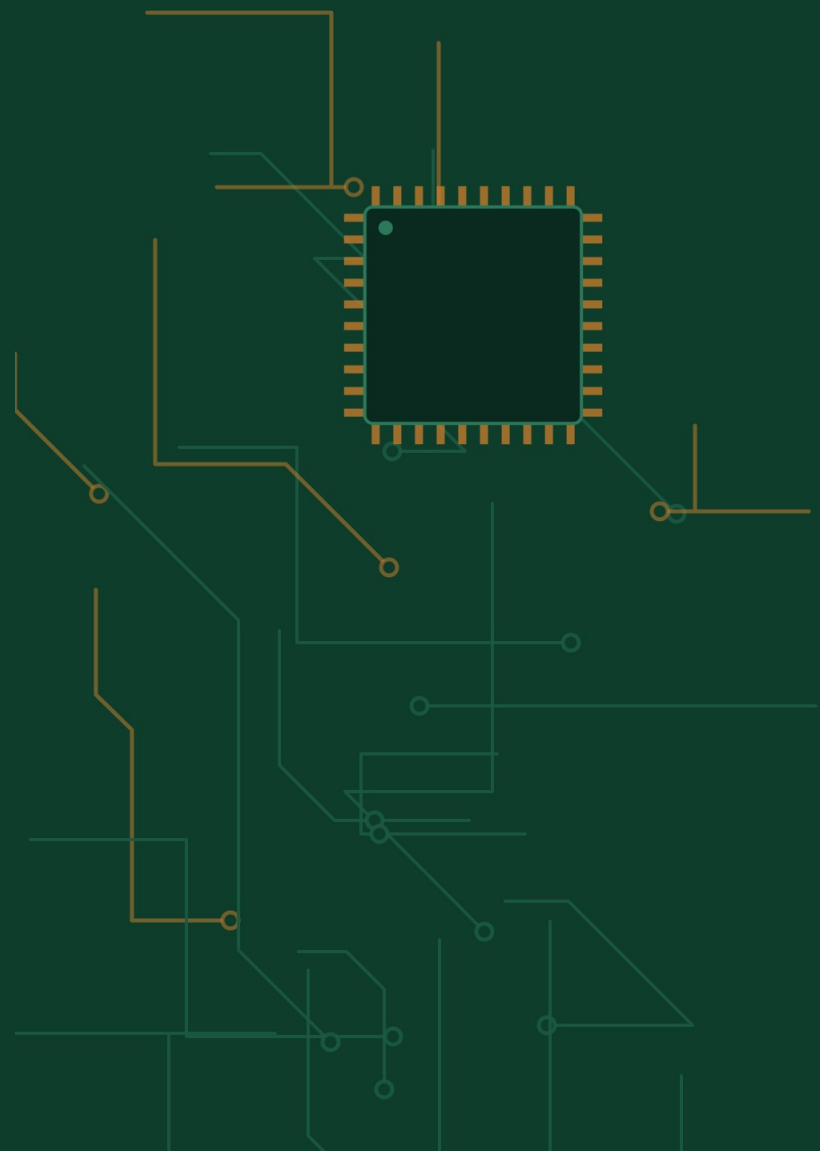
void loop() {
  portENTER_CRITICAL(&kunci);
  unsigned long n = jumlahTekan;
  jumlahTekan = 0;
  portEXIT_CRITICAL(&kunci);
  Serial.println(n);   // di luar
}
```

- Tanpa kunci, ISR bisa menyela antara salin dan nolkan
- Mematikan interupsi inti ini + memegang *spinlock*
- Di ISR pakai versi `_ISR`
- Harus **sangat pendek**: tanpa Serial, delay, FreeRTOS

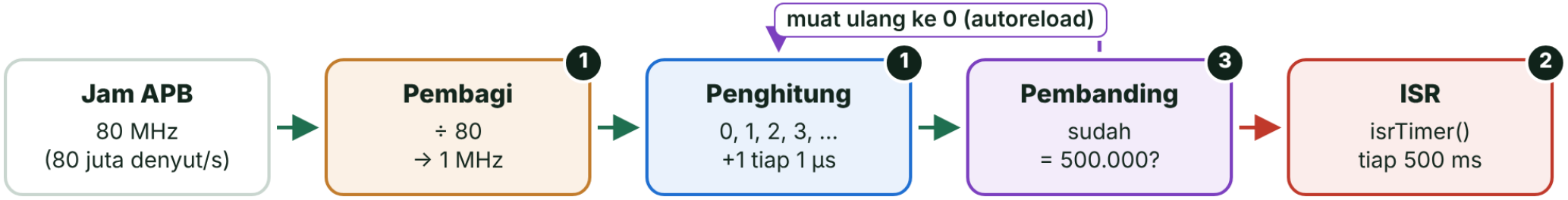
03

Timer Perangkat Keras

Alarm dapur di dalam cip



Cara Kerja Timer ESP32



1 `timerBegin(1000000)`

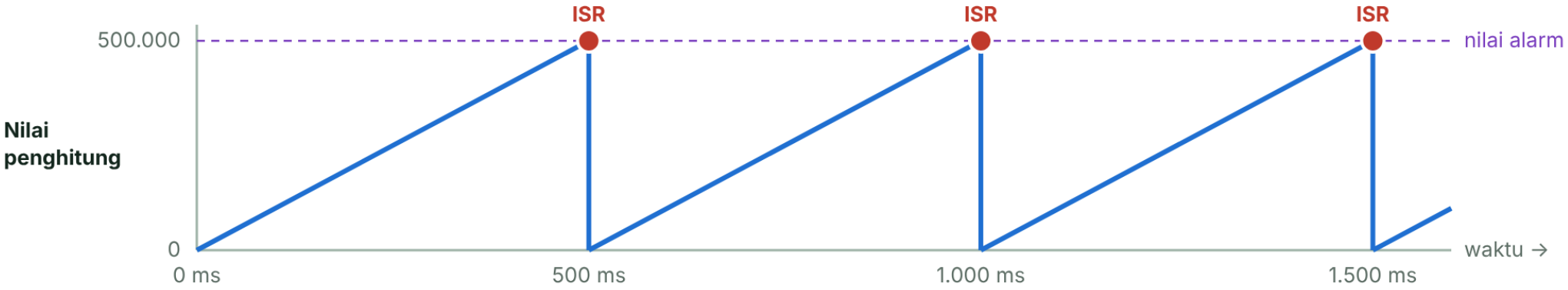
Buat timer yang menghitung 1 juta tik per detik (1 tik = 1 µs).

2 `timerAttachInterrupt(t, &isrTimer)`

Pasang fungsi ISR yang dipanggil saat alarm.

3 `timerAlarm(t, 500000, true, 0)`

Alarm di 500.000 tik, lalu muat ulang ke 0 dan ulangi.



Timer: Tiga Langkah API 3.x

kode/03-timer-hardware (disederhanakan)

```
hw_timer_t *timerKu = NULL;

void IRAM_ATTR isrTimer() {
  // singkat: balik LED, catat waktu
}

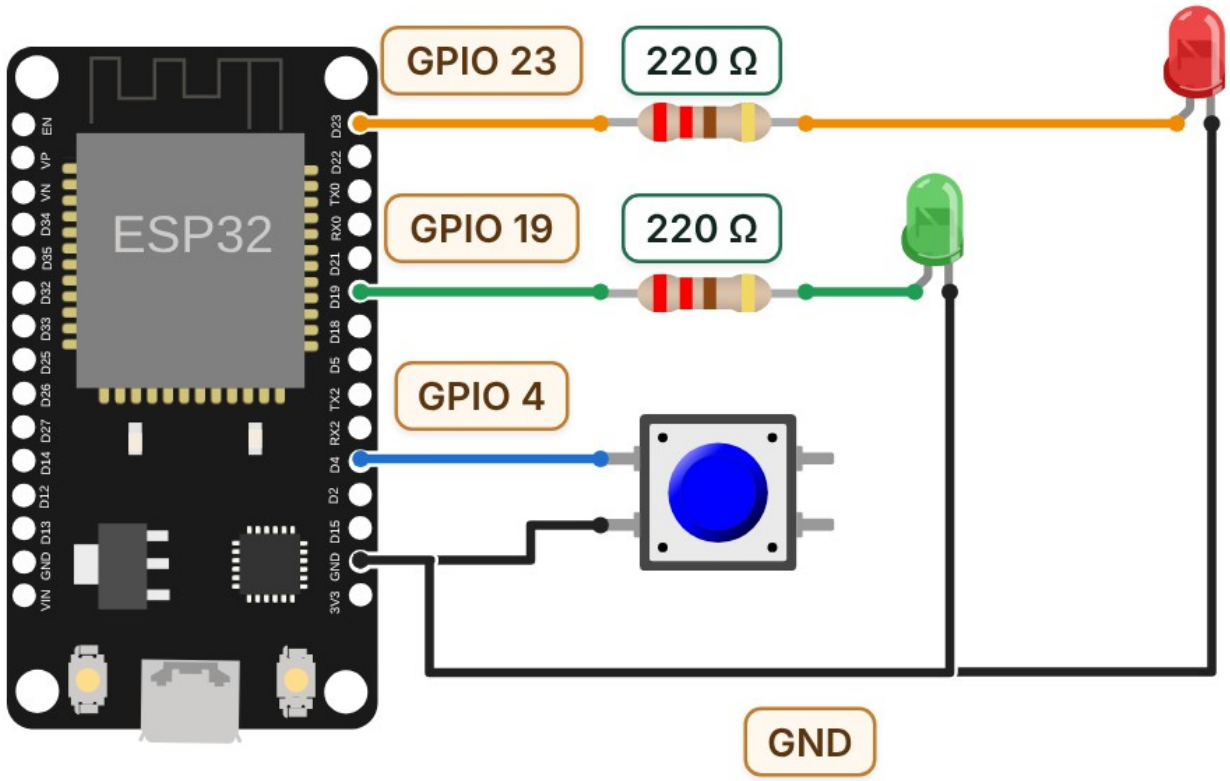
void setup() {
  // 1) 1 MHz → 1 tik = 1 µs
  timerKu = timerBegin(1000000);
  // 2) pasang ISR
  timerAttachInterrupt(timerKu, &isrTimer);
  // 3) alarm 500.000 tik, ulangi terus
  timerAlarm(timerKu, 500000, true, 0);
}
```

- Nilai alarm = frekuensi × selang
- `true` = berulang; `0` = tanpa batas
- Frekuensi ±1.221 Hz–40 MHz; di luar itu `NULL`
- API 2.x (`timerBegin(0, 80, true)`) tidak berlaku

Menghitung Nilai Alarm

Frekuensi timer	1 tik	Selang	Nilai alarm
1 MHz	1 µs	1 detik	1.000.000
1 MHz	1 µs	500 ms	500.000
1 MHz	1 µs	10 ms	10.000
10 kHz	100 µs	1 detik	10.000

Praktikum: Satu Rangkaian, Empat Program

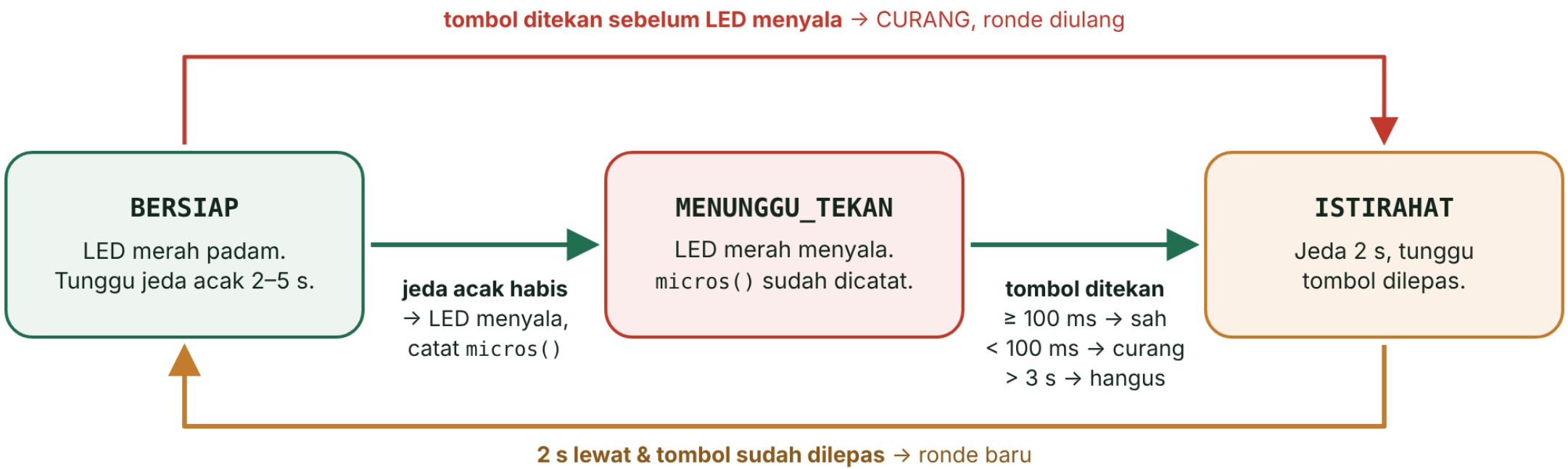


Gambar komponen: @wokwi/elements (MIT, © Uri Shaked)

Empat Praktikum

Praktikum	Konsep	Yang diamati
1. Dua LED tanpa delay()	Pola millis(), banyak tugas	LED beda irama, tombol tetap sigap
2. Interupsi tombol	ISR, volatile, bagian kritis	Tekanan sah vs interupsi mentah
3. Timer perangkat keras	timerBegin, timerAlarm	LED hijau tepat tiap 500 ms
4. Gim waktu reaksi	micros(), ISR, tahapan	Waktu reaksi dan deteksi curang

Praktikum 4: Tahapan Gim Reaksi



ISR `isrTombol()` hanya mencatat `micros()` saat tombol pertama kali ditekan. `loop()` membacanya di tahap BERSIAP (curang?) dan MENUNGGU_TEKAN (waktu reaksi).

Kuis Cepat



sebelumnya = 4.294.967.000 dan sekarang = 200 (unsigned long 32-bit). Berapa sekarang − sebelumnya?

A −4.294.966.800

B 200

C 496

D 4.294.967.200

Rangkuman

- `delay()` memblokir; pola `millis()` melirik jam tanpa menunggu
- Bandingkan **selisih** `unsigned long` → aman saat luapan (±49,7 hari)
- Interupsi: perangkat keras memanggil **ISR** pada tepi FALLING/RISING/CHANGE
- ISR singkat, `IRAM_ATTR`, `volatile`, bagian kritis, tanpa Serial/delay
- **Timer** 64-bit: `timerBegin` → `timerAttachInterrupt` → `timerAlarm`

Sebelum Pertemuan 6



Tugas

- Latihan 1: lampu lalu lintas tanpa delay()
- Latihan 3: debounce yang lebih kokoh



Persiapan

- Baca modul Pertemuan 6: Input Analog (ADC)
- Selesaikan Praktikum 3 dan 4 bila belum



Butuh Bantuan?

- Baca panduan/06-mengatasi-masalah.md
- Tanyakan di forum kelas